Figure 9.24 A perfectly regular k-d tree partitioning with square cells. On the left, we show what can be considered the average case, with the tree's leaf point at the center of the region: then the furthest the target could be, inside the region, is at half the distance of the square's diagonal. If we draw the circle circumscribed to the square, it intersects just he 4 regions adjacent to the sides of current one. On the right, an example of another, more generic and less optimistic case: even if the distance is larger than the average, the hypersphere centered at the target node only intersects 4 other regions. Of course, in the worst-case scenario, it might intersect up to 8 other regions.

The next step in the algorithm is back-tracking to visit all the regions that are within that distance from the target point. If all the regions are equally-sized and regularly shaped, this means that any region within distance must be in one of the neighboring rectangles (with respect to our leaf's region), and from geometry we know that in such a situation, with regular equally-sized rectangles that can be approximated to a rectangular grid, each rectangle has 8 neighbors. From figure 9.24, however, it is possible to see how, on average, even if potentially we would have to traverse at most 8 more branches, it's likely we only have to check 4 of them, because only the ones adjacent to current region's sides will be within distance $\sqrt{1/n}$. Hence, this makes the total average running time `O(4*log(n))`.